

tkinter基礎知識ガイド

Pythonのデスクトップアプリ用ライブラリであるtkinterについて、最低限の基礎知識を解説するドキュメントです。

さらに詳しく知りたい・理解したいという方は「tkinter 公式ドキュメント」を参照してください。

詳細な仕様とコーディング例を確認できます。

Pythonは海外発祥のプログラミング言語になりますが、公式ドキュメントは日本語に翻訳されているため読みやすく設計されています。

▼tkinter 公式ドキュメント▼

<https://docs.python.org/ja/3.13/library/tkinter.html>

<目次>

1. tkinterとは

- 1.1. tkinterの特徴
- 1.2. 基本的な使い方
- 1.3. 代表的なウィジェット
- 1.4. レイアウト管理
- 1.5. イベントとコールバック
- 1.6. カスタマイズと拡張
- 1.7. tkinterの構造(オブジェクト指向的設計)
- 1.8. tkinterを扱う際の注意点
- 1.9. tkinter製デスクトップアプリケーション例

1. tkinterとは

tkinterは、Pythonに標準で搭載されているGUI(Graphical User Interface)ライブラリです。PythonからTcl/TkというGUIツールキットを使えるようにしたインターフェースです。ボタンやラベル/テキストボックスなどウィジェットを使い、デスクトップアプリケーションを作成できます。Python3.xではimport tkinterで利用します(Python2.x:import Tkinter)。

1.1 tkinterの特徴

以下は、tkinterの特徴になります。

▼tkinterの特徴

特徴	内容
標準ライブラリ	Python標準搭載、追加インストール不要
クロスプラットフォーム	WindowsOS/macOS/Linuxで動作
シンプルな記述	少量のコードでGUI作成可能
豊富なウィジェット	多彩なウィジェット提供
イベント駆動型	イベントに応じた処理可能

特にPython初学者の場合、初めてのデスクトップ用ライブラリとしてtkinterは利用しやすく学ぶ上でもおすすめです。

1.2. 基本的な使い方

ここでは、最小構成のGUIアプリを記載します。

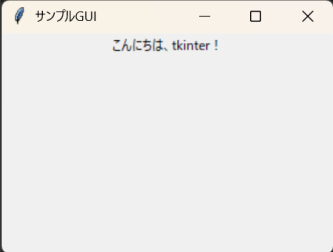
▼最小構成のGUIアプリ例

```
import tkinter as tk

root = tk.Tk()          # メインウィンドウの作成
root.title("サンプルGUI") # タイトル設定
root.geometry("300x200") # ウィンドウサイズ設定

label = tk.Label(root, text="こんにちは、tkinter!")
label.pack()

root.mainloop()        # イベントループ開始
```

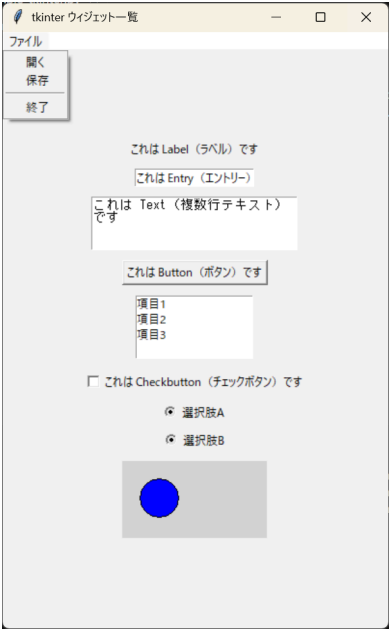


1.3. 代表的なウィジェット

以下は、tkinterで代表的なウィジェットになります。

▼tkinterの代表的なウィジェット一覧

ウィジェット	概要
Label	テキスト表示
Button	ボタン
Entry	一行テキスト入力欄
Text	複数行テキスト入力欄
Frame	ウィジェットのグループ化
Listbox	リスト表示
Checkbox	チェックボックス
Radiobutton	ラジオボタン
Canvas	描画領域(図形や画像表示)
Menu	メニューバー



1.4. レイアウト管理

tkinterには3種類のレイアウト(ウィジェット配置)方法があります。

▼レイアウト方法

方法	説明
.pack()	自動的に上下または左右に詰めて配置
.grid()	行・列を指定して配置 表形式のレイアウトに最適
.place()	ピクセル単位での絶対位置指定

.grid()によるレイアウト方法は行列を指定し配置するため、ウィンドウサイズに合わせ表形式のレイアウトが実現できておすすめです。

1.5. イベントとコールバック

GUIは「イベント駆動型」で動作します。ユーザーの操作(クリック/入力/キー押下など)に応じて関数を呼び出すのが基本的な流れです。

▼イベントとコールバックの例

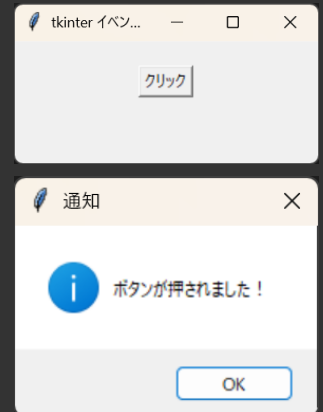
```
import tkinter as tk
from tkinter import messagebox

# ウィンドウ作成
root = tk.Tk()
root.title("tkinter イベントとコールバック")
root.geometry("250x100")

# ボタンクリック時の処理
def on_click():
    messagebox.showinfo("通知", "ボタンが押されました！")

# ボタンの設置
button = tk.Button(root, text="クリック", command=on_click)
button.pack(padx=20, pady=20)

# メインループ
root.mainloop()
```



▼messagebox

tkinter.messagebox は情報/警告/エラー/確認など
ポップアップダイアログを表示するモジュール

▼messageboxの関数例

showinfo(title, message)	情報メッセージ
showwarning(title, message)	警告メッセージ
showerror(title, message)	エラーメッセージ
askyesno(title, message)	はい／いいえの確認(True/Falseを返す)

様々な形で関数を作成することで、メッセージボックス以外のモジュールも利用することができます。

1.6. カスタマイズと拡張

tkinterのみのデスクトップアプリ開発には、デザイン性やモダンなウィジェットに限界があります。そのため、以下の方法でカスタマイズと拡張が可能になります。

▼カスタマイズと拡張方法

カスタマイズと拡張について
フォントや色、サイズの変更が可能
ttkモジュール(tkinter.ttk)を使うことでモダンな外観のウィジェット利用可能
外部ライブラリ(ttkbootstrapなど)を使えばBootstrap風のテーマにも対応

デスクトップアプリ開発にて、モダンなデザイン性を求める場合はttkやttkbootstrapを利用すると実現しやすくなります。

1.7. tkinterの構造(オブジェクト指向的設計)

tkinterでは、アプリ全体やウィジェットをオブジェクト指向で構築できます。

▼クラスベースの設計例

```
class MyApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("クラスベースのアプリ")
        self.geometry("300x200")
        label = tk.Label(self, text="クラスベースで作成")
        label.pack()

app = MyApp()
app.mainloop()
```

クラスに関する詳細な利用方法については、以下のドキュメントを参照してください。

▼Python基礎知識ガイド▼
無料配布のフォルダ内にて確認してください

1.8. tkinterを扱う際の注意点

tkinterによるアプリ開発時に、よくある注意点を記載しておきます。

▼tkinterを扱う際の注意点

<code>.mainloop()</code> は1回だけ呼び出すこと
レイアウト方法(pack, grid, place)は混在させない
ウィジェットの変化を反映したい場合は <code>.update()</code> を使う

1.9. tkinter製デスクトップアプリケーション例

開発者次第で様々なアプリが開発できますが、Python初学者であり開発経験が浅い場合、簡単なアプリケーション開発から実施することをおすすめします。

▼tkinter製デスクトップアプリケーション例

ToDoアプリ
高機能(関数)電卓
フォーム入力アプリ(CSV保存など)
画像取り込み(OCR)アプリ
Webスクレイピングアプリ

特に、デスクトップアプリケーションの魅力は、GUI操作によるプログラムを前提に開発できることです。

そのため、ユーザー操作による様々なイベントを元にプログラムの実装を考えられます。Pythonを理解しtkinterを利用することで一部の業務効率化や自動化を実現できます。